CSA1708-ARTIFICIAL INTELLIGENCE
LAB EXPERIMENTS

Program 1: Python Program to Solve the 8-Puzzle Problem

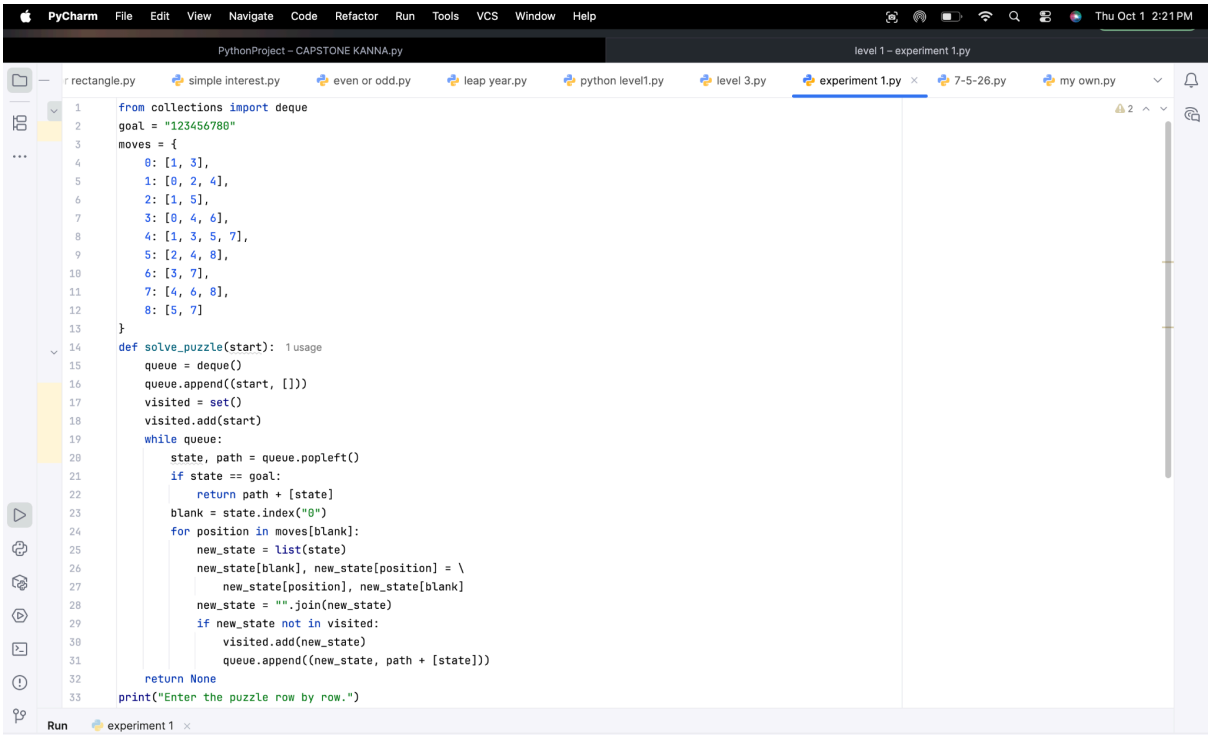
AIM:

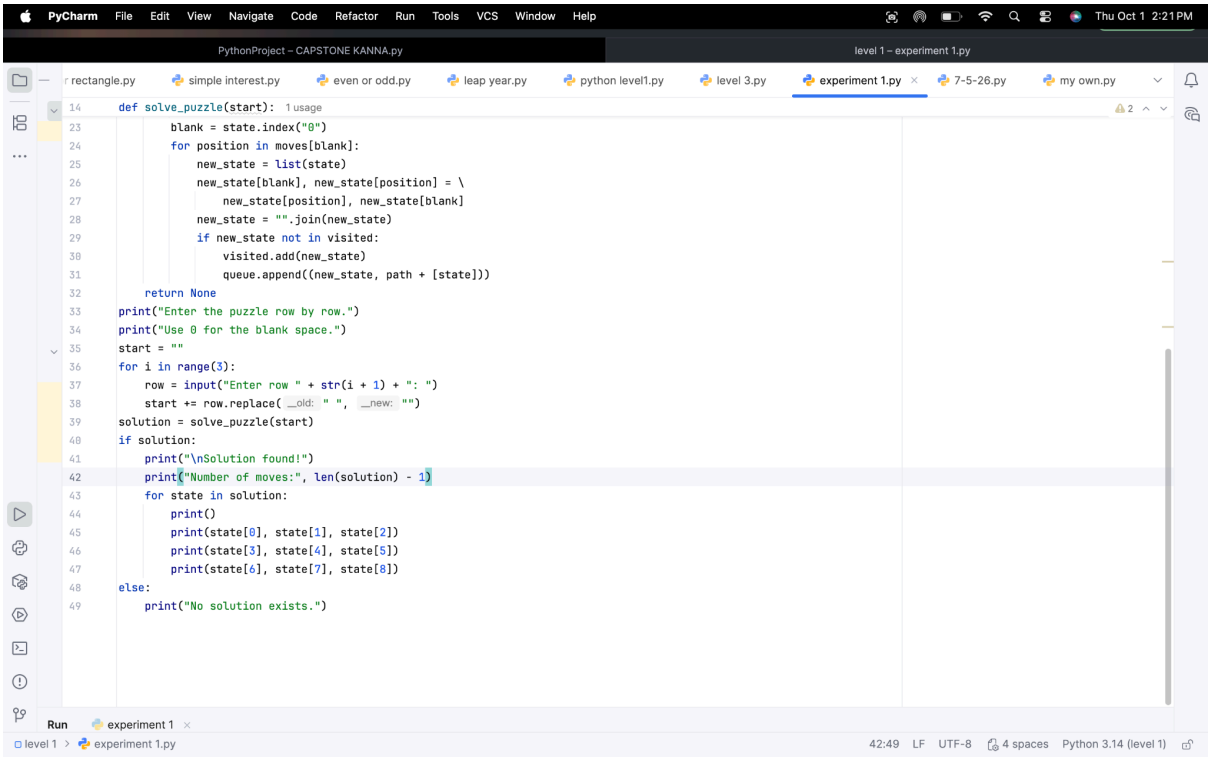
To write and execute a Python program to solve the 8-Puzzle problem using the Breadth-First Search (BFS) technique.

ALGORITHM:

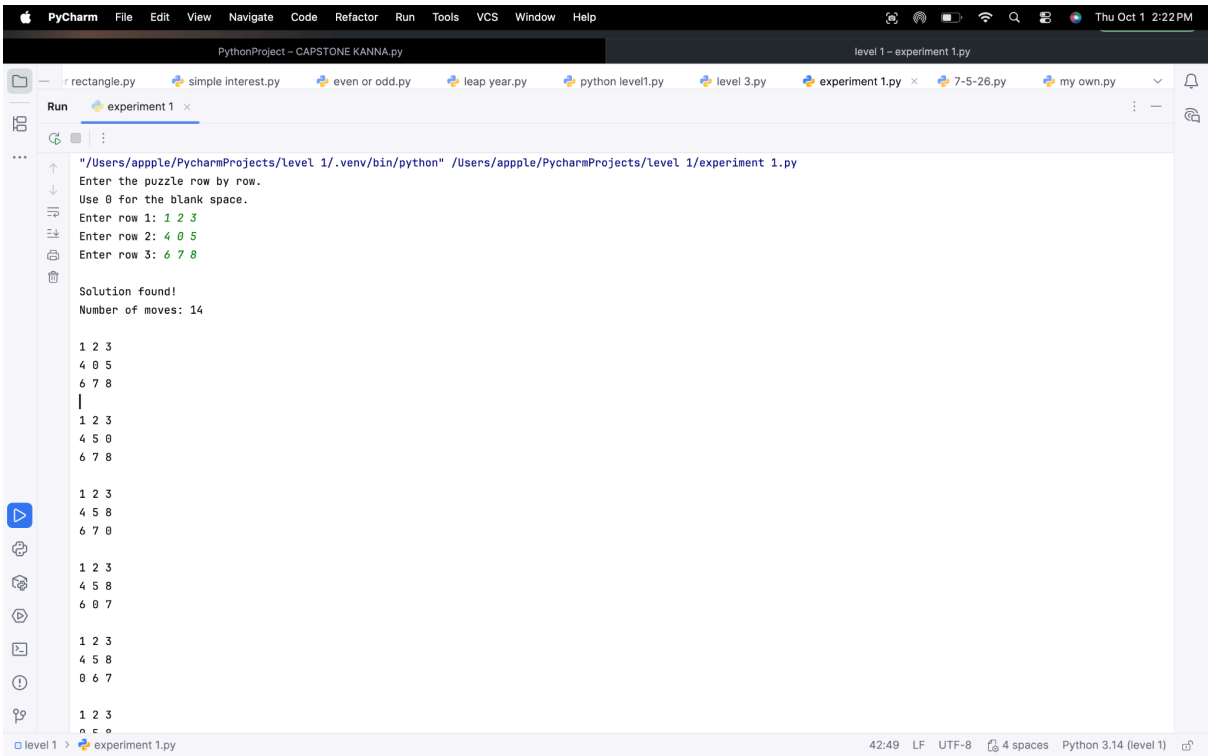
1. Start the program.
2. Define the goal state as 123456780, where 0 represents the blank space.
3. Define all possible movements of the blank space.
4. Store the initial state in a queue.
5. Remove the first state from the queue.
6. Check whether the current state is the goal state.
7. If it is the goal state, display the solution.
8. Otherwise, move the blank space to all possible positions.
9. Add the new states to the queue if they have not been visited.
10. Repeat the process until the goal state is reached.
11. Stop the program.

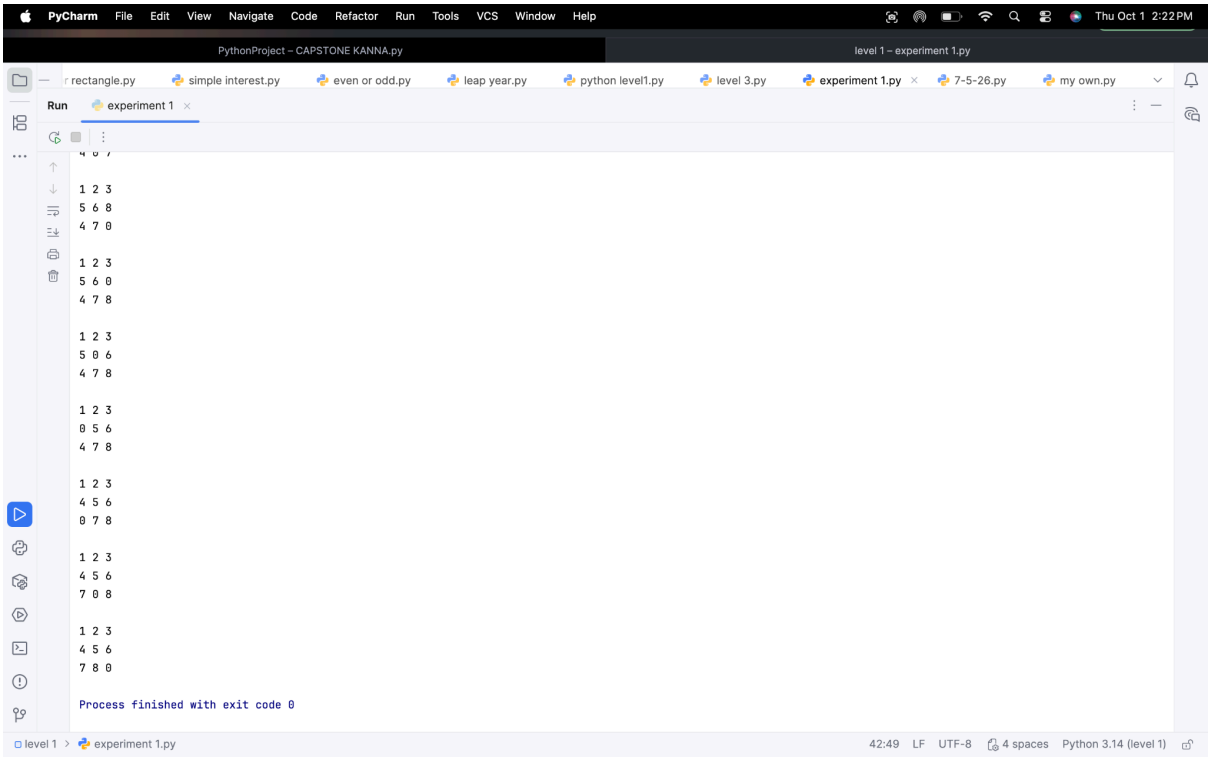
CODE:




PyCharm File Edit View Navigate Code Refactor Run Tools VCS Window Help
Thu Oct 1 2:21 PM
PythonProject – CAPSTONE KANNA.py
level 1 – experiment 1.py
rectangle.py
simple interest.py
even or odd.py
leap year.py
python level1.py
level 3.py
experiment 1.py
7-5-26.py
my own.py
def solve_puzzle(start): 1 usage
blank = state.index("0")
for position in moves[blank]:
new_state = list(state)
new_state[blank], new_state[position] = \
new_state[position], new_state[blank]
new_state = "".join(new_state)
if new_state not in visited:
visited.add(new_state)
queue.append((new_state, path + [state]))
return None
print("Enter the puzzle row by row.")
print("Use 0 for the blank space.")
start = ""
for i in range(3):
row = input("Enter row " + str(i + 1) + ": ")
start += row.replace(__old: " ", __new: "")
solution = solve_puzzle(start)
if solution:
print("\nSolution found!")
print("Number of moves:", len(solution) - 1)
for state in solution:
print()
print(state[0], state[1], state[2])
print(state[3], state[4], state[5])
print(state[6], state[7], state[8])
else:
print("No solution exists.")
Run
experiment 1
level 1
experiment 1.py
42:49 LF UTF-8 4 spaces Python 3.14 (level 1)

OUTPUT:


PyCharm File Edit View Navigate Code Refactor Run Tools VCS Window Help
Thu Oct 1 2:22 PM
PythonProject – CAPSTONE KANNA.py
level 1 – experiment 1.py
Run
experiment 1
"/Users/appple/PycharmProjects/level 1/.venv/bin/python" /Users/appple/PycharmProjects/level 1/experiment 1.py
Enter the puzzle row by row.
Use 0 for the blank space.
Enter row 1: 1 2 3
Enter row 2: 4 0 5
Enter row 3: 6 7 8
Solution found!
Number of moves: 14
1 2 3
4 0 5
6 7 8
1 2 3
4 5 0
6 7 8
1 2 3
4 5 8
6 7 0
1 2 3
4 5 8
6 0 7
1 2 3
4 5 8
0 6 7
1 2 3
level 1
experiment 1.py
42:49 LF UTF-8 4 spaces Python 3.14 (level 1)



RESULT: Thus, the 8-Puzzle problem was successfully solved using the Breadth-First Search (BFS) algorithm, and the sequence of states from the initial state to the goal state was obtained.

Program 2: Python program to solve the 8-Queen Problem.

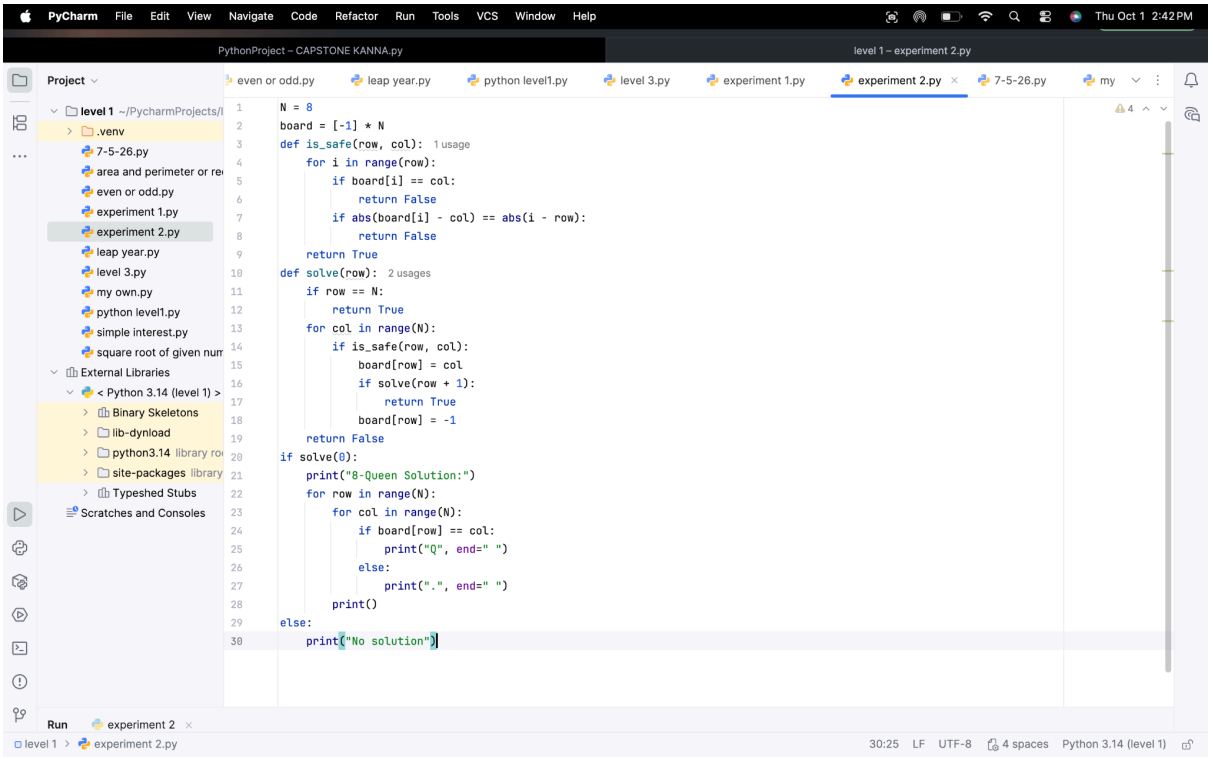
AIM :

To write and execute a Python program to solve the **8-Queen problem** using the **Backtracking technique**.

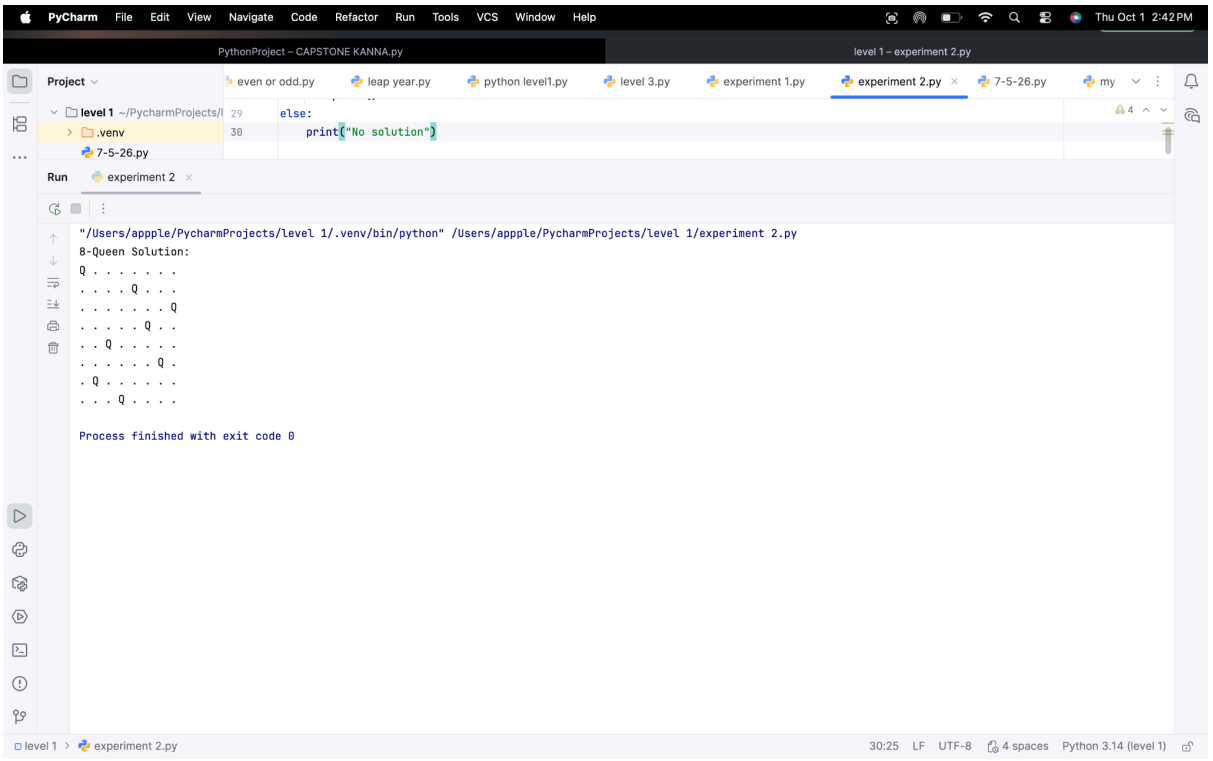
ALGORITHM:

1. Start the program.
2. Create an 8×8 chessboard.
3. Place a queen in the first row.
4. Check whether the selected position is safe.
5. A position is safe if no other queen is in the same column or diagonal.
6. If the position is safe, place the queen.
7. Move to the next row.
8. If no safe position is available, go back to the previous row.
9. Change the previous queen's position and try another position.
10. Continue until all 8 queens are placed safely.
11. Display the chessboard.
12. Stop the program.

CODE:



OUTPUT:



RESULT: Thus, the 8-Queen problem was successfully solved using the Backtracking technique, and a valid arrangement of 8 queens was obtained such that no two queens attack each other.

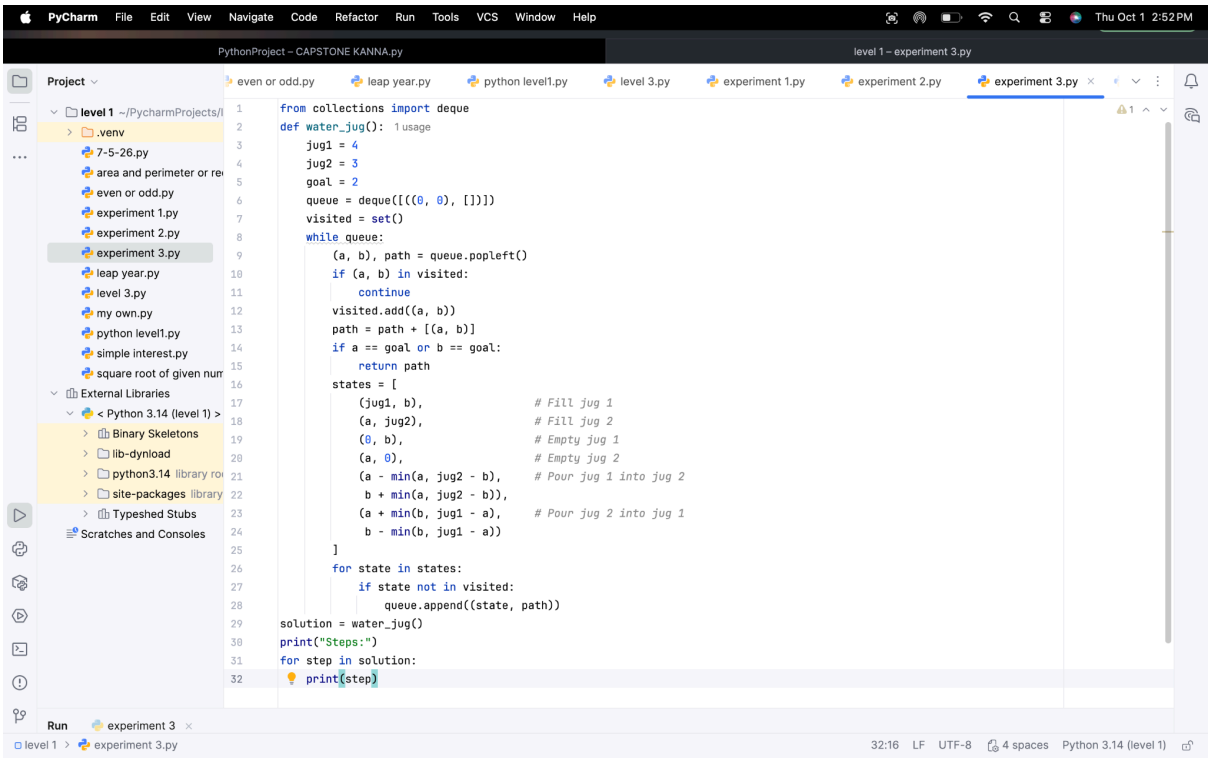
PROGRAM 3: Python program for the Water Jug Problem.

AIM: To write and execute a Python program to solve the Water Jug problem using the Breadth First Search (BFS) technique.

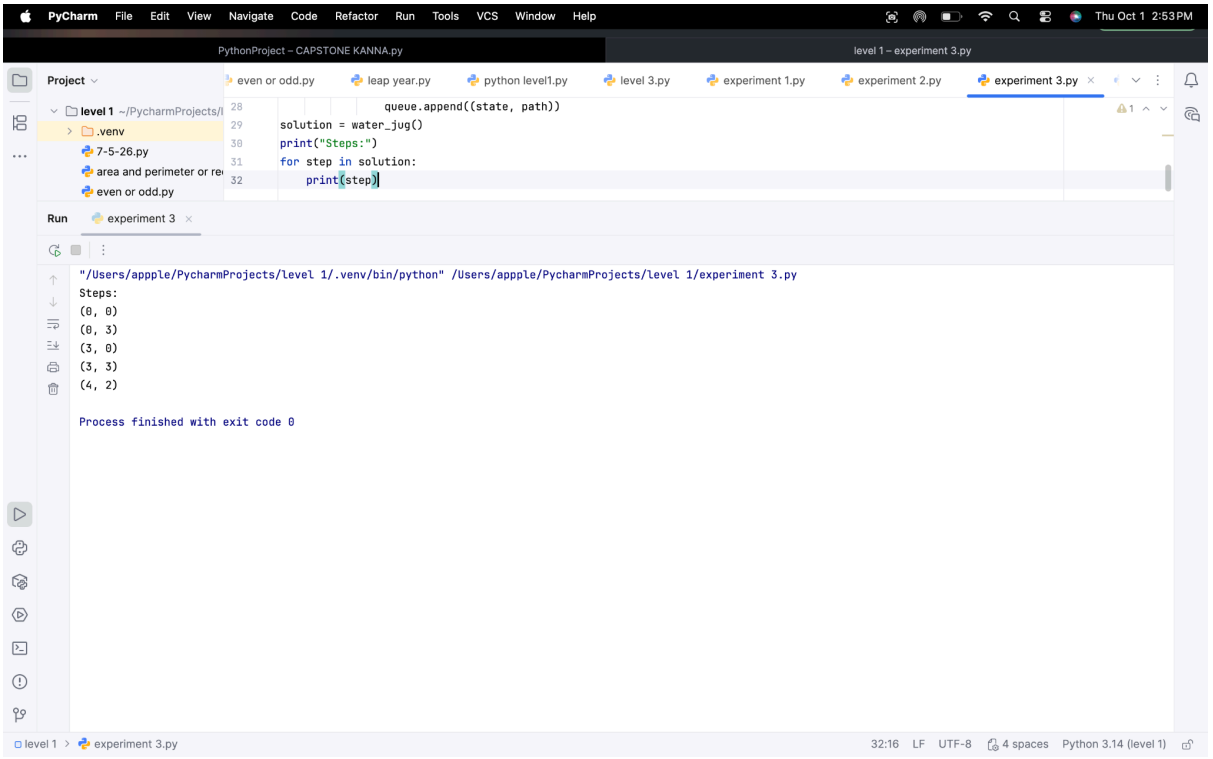
ALGORITHM:

1. Start the program.
2. Define the capacities of the two jugs.
3. Set the initial state of both jugs to empty.
4. Define the required goal amount.
5. Store the initial state in a queue.
6. Generate possible operations:
    - Fill a jug.
    - Empty a jug.
    - Pour water from one jug into another.
7. Add the new states to the queue.
8. Avoid states that have already been visited.
9. Check whether the required amount of water has been obtained.
10. If the goal is reached, display all the steps.
11. Otherwise, continue generating new states.
12. Stop the program.

CODE:



OUTPUT:



RESULT:

Thus, the Water Jug problem was successfully solved using the Breadth First Search (BFS) technique, and the required quantity of water was obtained through a sequence of valid operations.

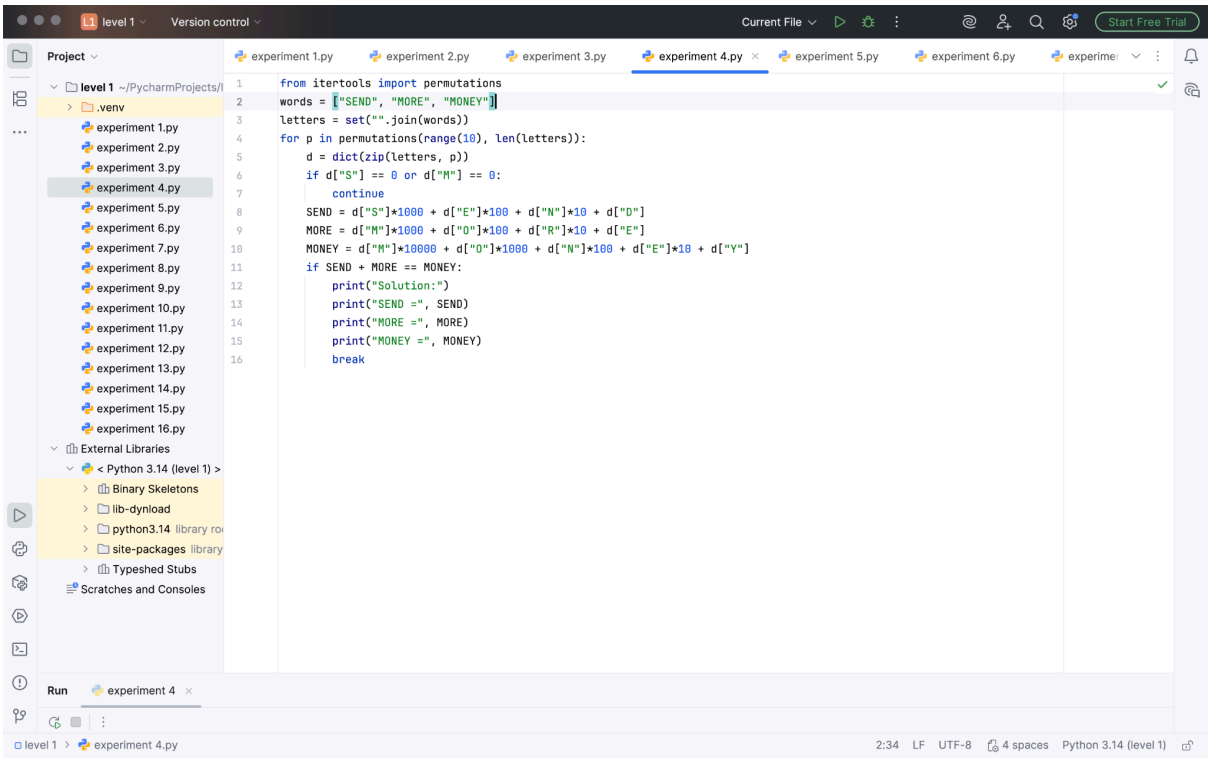
PROGRAM 4: Python program for Cript-Arithmetic problem.

AIM: To write a Python program to solve a crypt-arithmetic problem by assigning unique digits to letters such that the given arithmetic equation is satisfied.

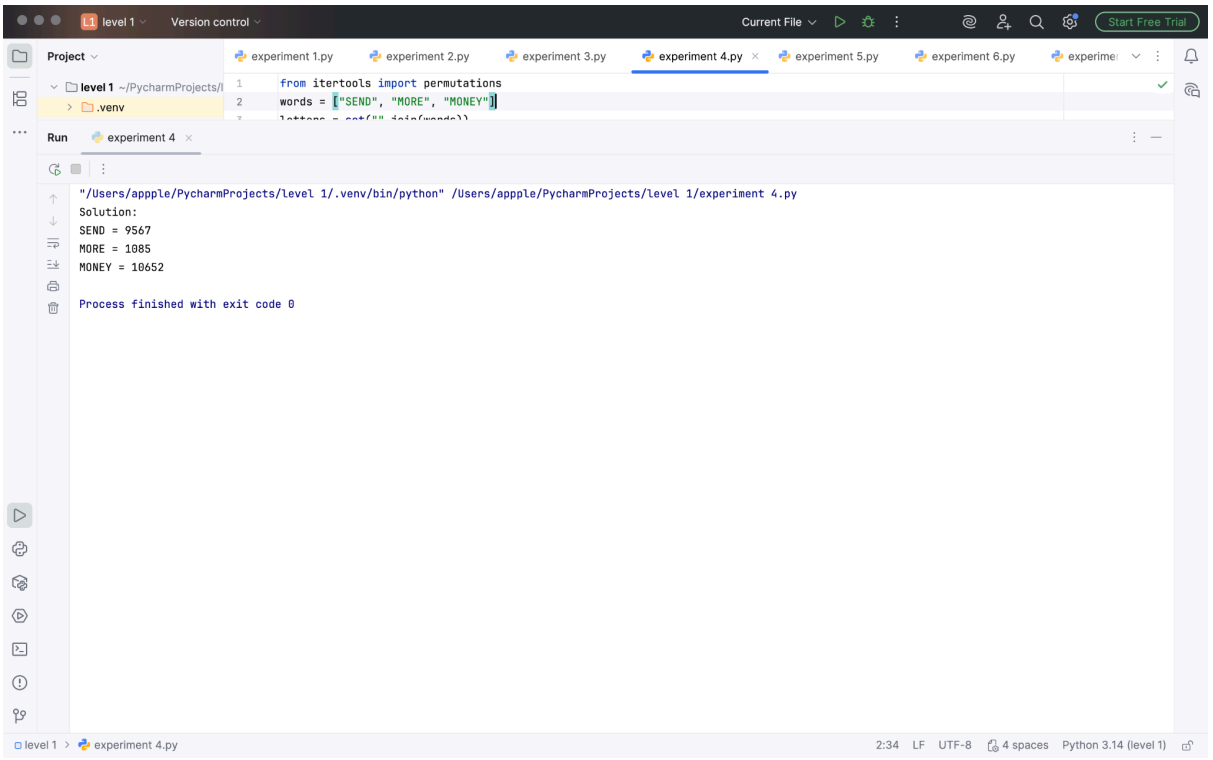
ALGORITHM:

1. Start the program.
2. Define the words used in the crypt-arithmetic problem.
3. Identify all unique letters in the words.
4. Assign different digits (0–9) to each letter using permutations.
5. Ensure that the first letter of each word is not zero.
6. Check whether the arithmetic equation is satisfied.
7. Display the valid digit assignments and the corresponding equation.
8. Stop the program.

CODE:



OUTPUT:



RESULT:

Thus, the Python program to solve the crypt-arithmetic problem was executed successfully, and the solution SEND + MORE = MONEY was obtained as 9567 + 1085 = 10652.

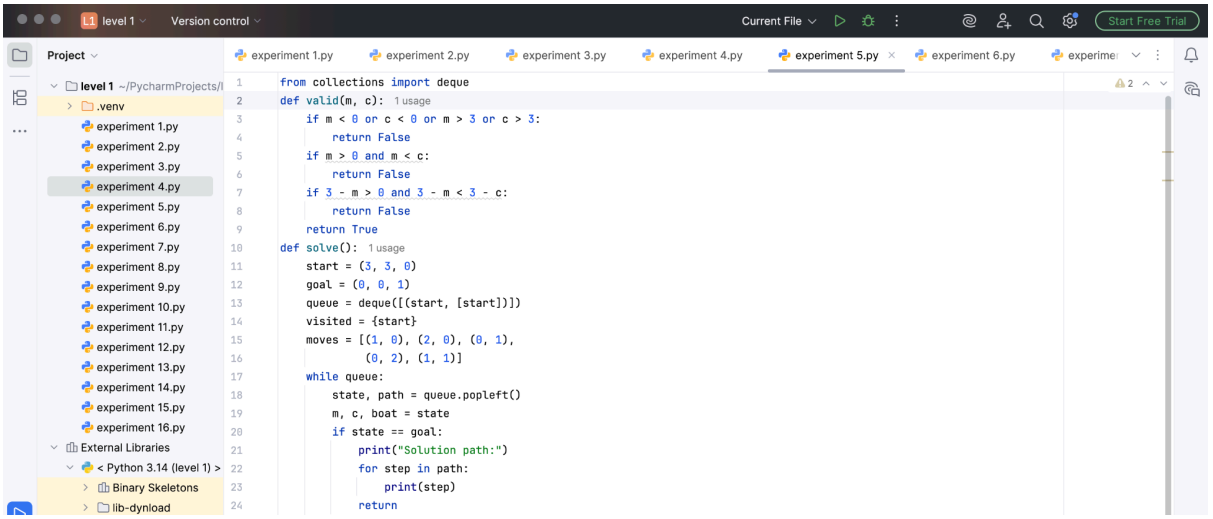
PROGRAM 5: Python program for Missionaries Cannibal problem

AIM: To write a Python program to solve the Missionaries and Cannibals problem using Breadth-First Search (BFS).

ALGORITHM:

1. Start the program.
2. Initialize three missionaries and three cannibals on the left bank of the river.
3. Define a boat that can carry one or two people at a time.
4. Ensure that cannibals never outnumber missionaries on either bank when missionaries are present.
5. Use BFS to explore all possible valid states.
6. Store visited states to avoid repetition.
7. Stop when all missionaries and cannibals reach the right bank.
8. Display the solution path and stop.

CODE:



```python
def solve():
    start = (3, 3, 0)
    goal = (0, 0, 1)
    queue = deque([[start, [start]]])
    visited = {start}
    moves = [(1, 0), (2, 0), (0, 1),
             (0, 2), (1, 1)]
    while queue:
        state, path = queue.popleft()
        m, c, boat = state
        if state == goal:
            print("Solution path:")
            for step in path:
                print(step)
            return
        for dm, dc in moves:
            if boat == 0:
                new_state = (m - dm, c - dc, 1)
            else:
                new_state = (m + dm, c + dc, 0)
            if valid(new_state[0], new_state[1]):
                if new_state not in visited:
                    visited.add(new_state)
                    queue.append(
                        (new_state, path + [new_state])
                        )
    print("No solution found")
solve()
```

Output:

```
"/Users/appple/PycharmProjects/level 1/.venv/bin/python" /Users/appple/PycharmProjects/level 1/experiment 5.py
Solution path:
(3, 3, 0)
(3, 1, 1)
(3, 2, 0)
(3, 0, 1)
(3, 1, 0)
(1, 1, 1)
(2, 2, 0)
(0, 2, 1)
(0, 3, 0)
(0, 1, 1)
(1, 1, 0)
(0, 0, 1)

Process finished with exit code 0
```

RESULT:

Thus, the Python program using BFS successfully solves the Missionaries and Cannibals problem by finding a valid sequence of boat crossings that transfers all three missionaries and three cannibals safely to the right bank.

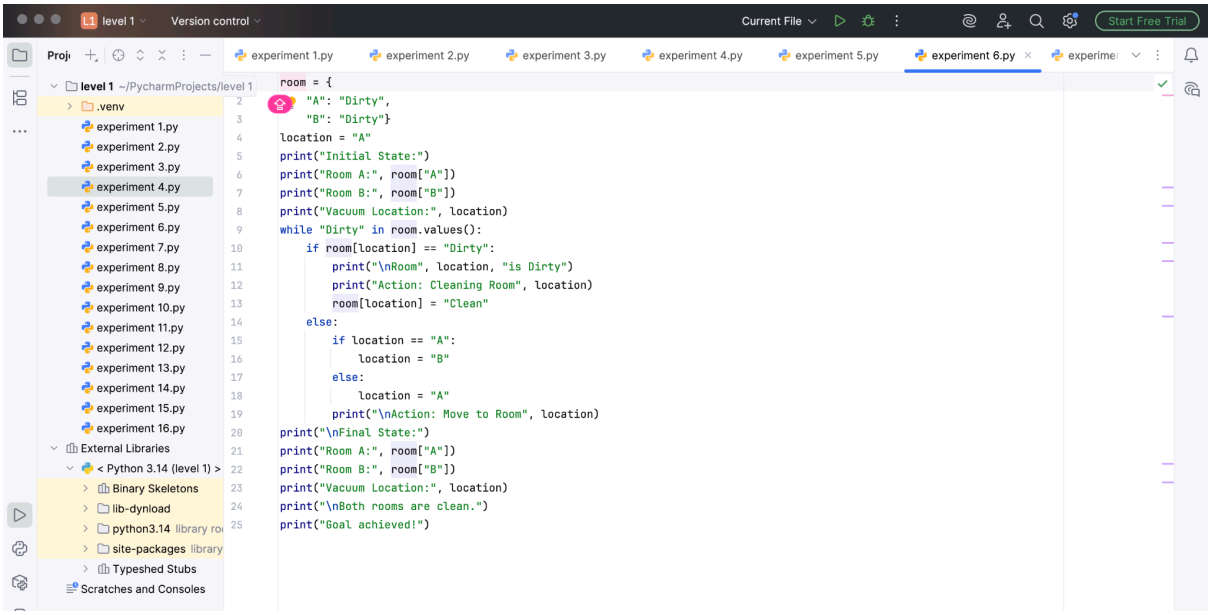
PROGRAM 6: python program for Vacuum Cleaner problem

AIM: To write a Python program to implement the Vacuum Cleaner problem using an intelligent agent.

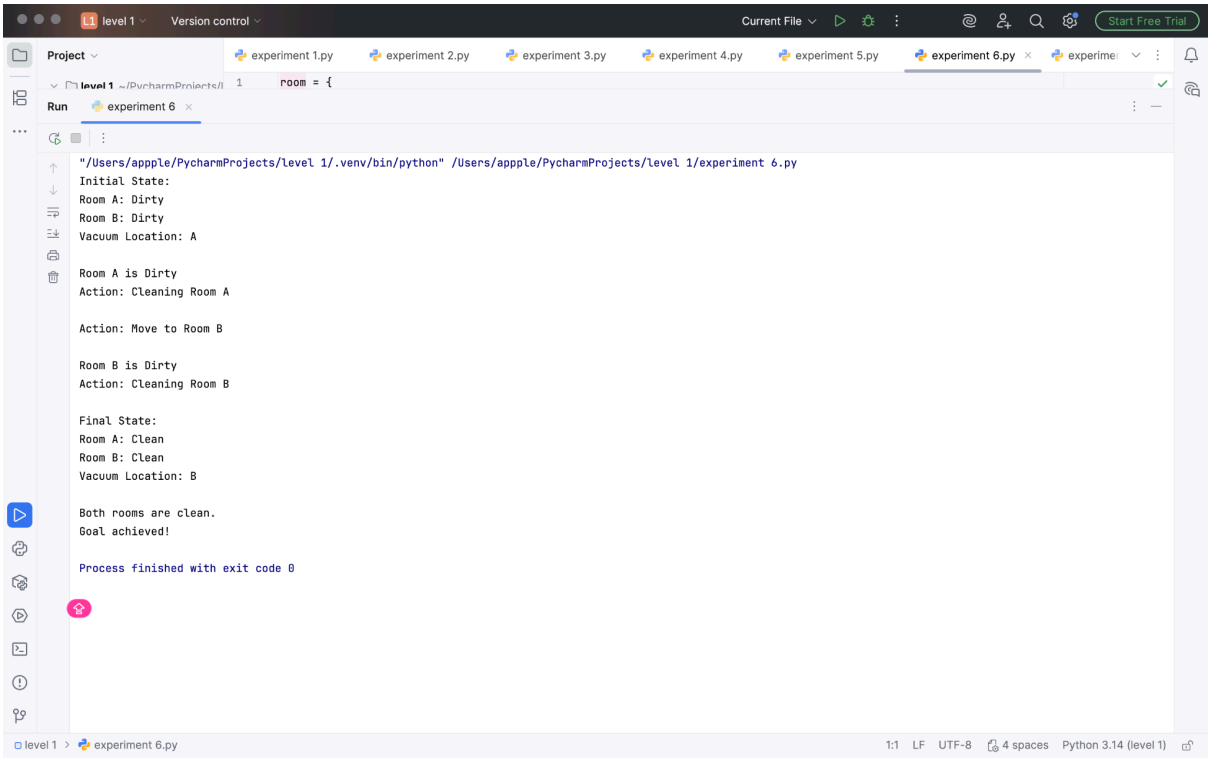
ALGORITHM:

1. Start the program.
2. Initialize two rooms, A and B, as either Clean or Dirty.
3. Place the vacuum cleaner in room A.
4. Check the current room's status.
5. If the room is dirty, clean it.
6. Move to the next room and repeat the process.
7. Stop when both rooms are clean.
8. Display the final status of the rooms.

CODE:



OUTPUT:



RESULT:

Thus, the Python program successfully implements the Vacuum Cleaner problem and cleans both rooms using a simple intelligent agent.

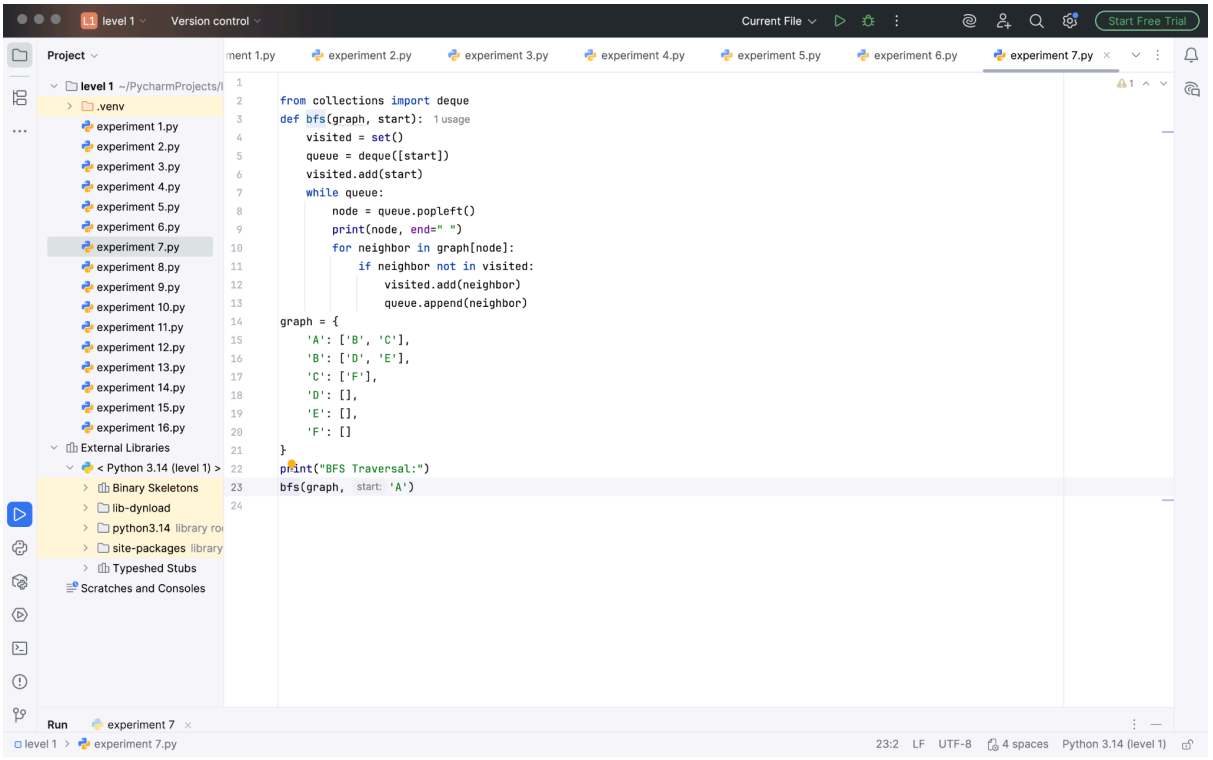
PROGRAM 7: python program to implement BFS.

AIM: To write a Python program to implement the Breadth-First Search (BFS) algorithm to traverse a graph.

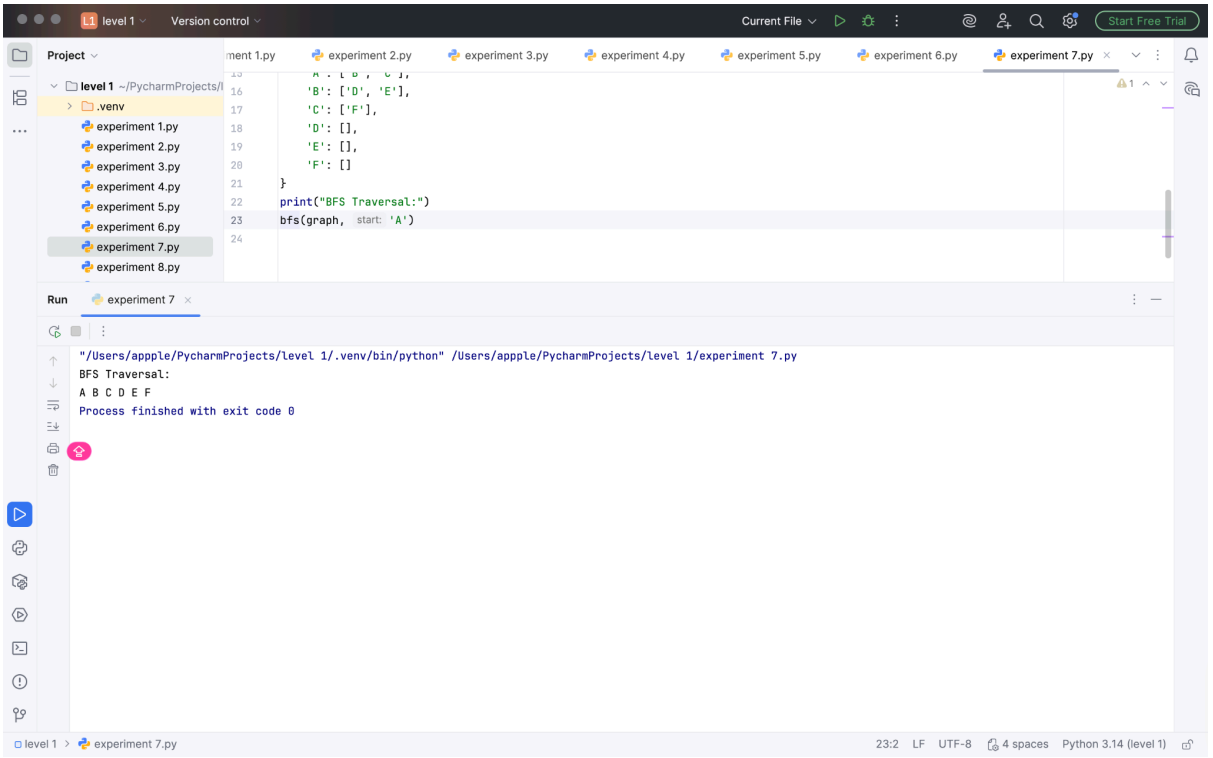
Algorithm

1. Start the program.
2. Create a graph using an adjacency list.
3. Initialize a queue with the starting node.
4. Mark the starting node as visited.
5. Remove a node from the front of the queue and print it.
6. Add all unvisited adjacent nodes to the queue and mark them as visited.
7. Repeat until the queue becomes empty.
8. Stop the program.

CODE:



OUTPUT:



RESULT:

Thus, the Python program successfully implements the Breadth-First Search algorithm and traverses the graph level by level, starting from node A.

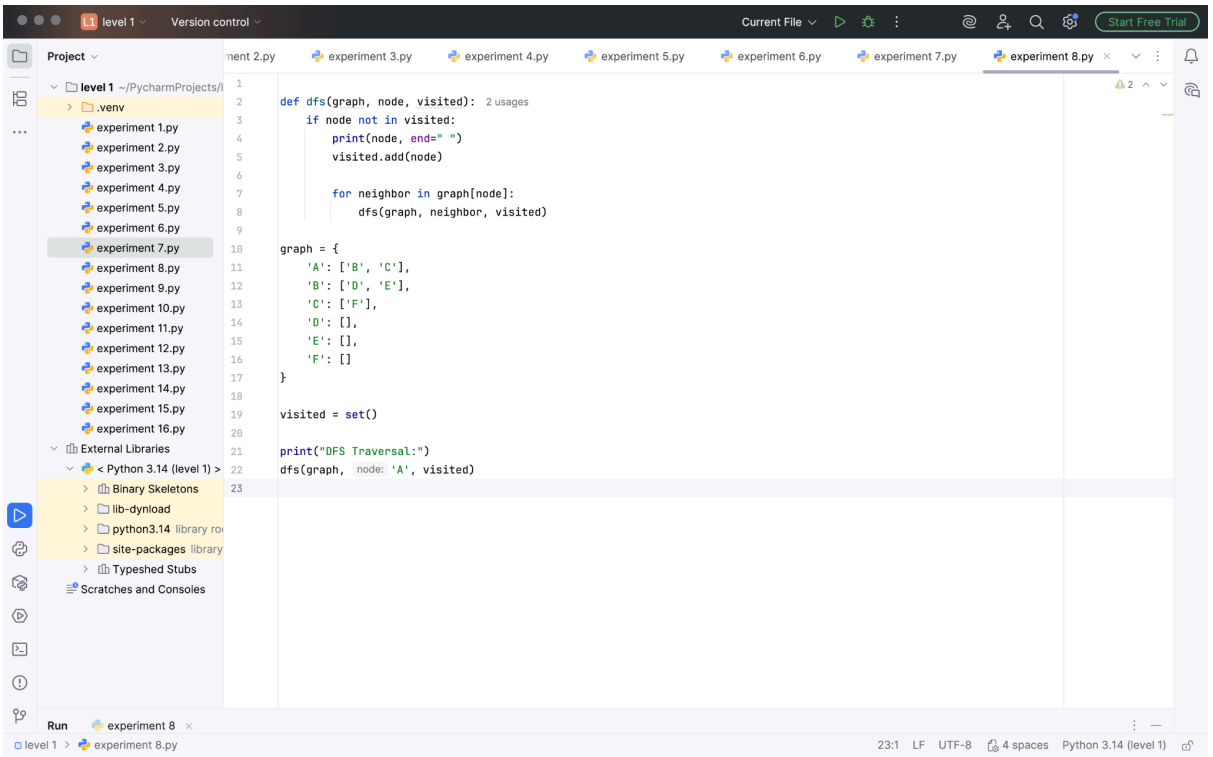
PROGRAM 8: Python program to implement DFS.

Aim: To write a Python program to implement the Depth-First Search (DFS) algorithm to traverse a graph.

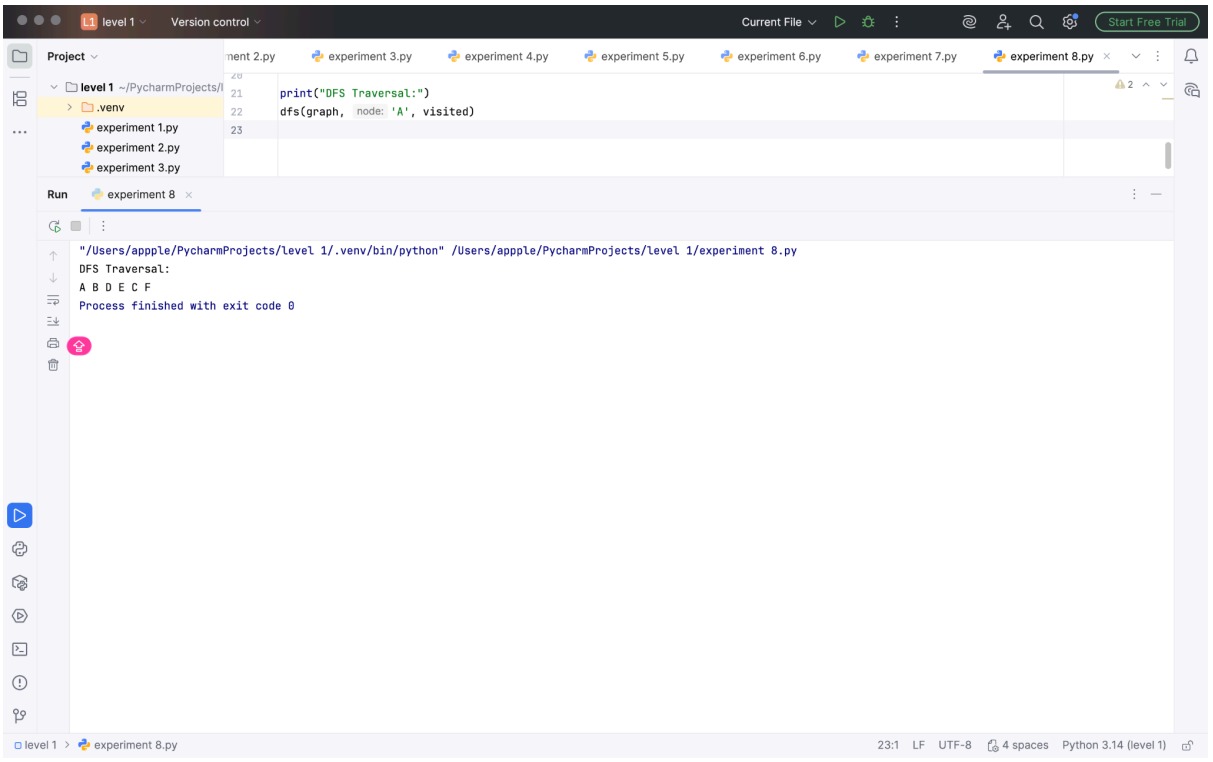
Algorithm:

1. Start the program.
2. Create a graph using an adjacency list.
3. Initialize an empty set to store visited nodes.
4. Start traversal from the given starting node.
5. Mark the current node as visited and print it.
6. Visit each unvisited adjacent node recursively.
7. Continue until all reachable nodes are visited.
8. Stop the program.

CODE:



OUTPUT:



RESULT: Thus, the Python program successfully implements the Depth-First Search algorithm and traverses the graph by exploring each branch deeply before moving to the next branch.

PROGRAM 9: Python to implement Travelling Salesman Problem

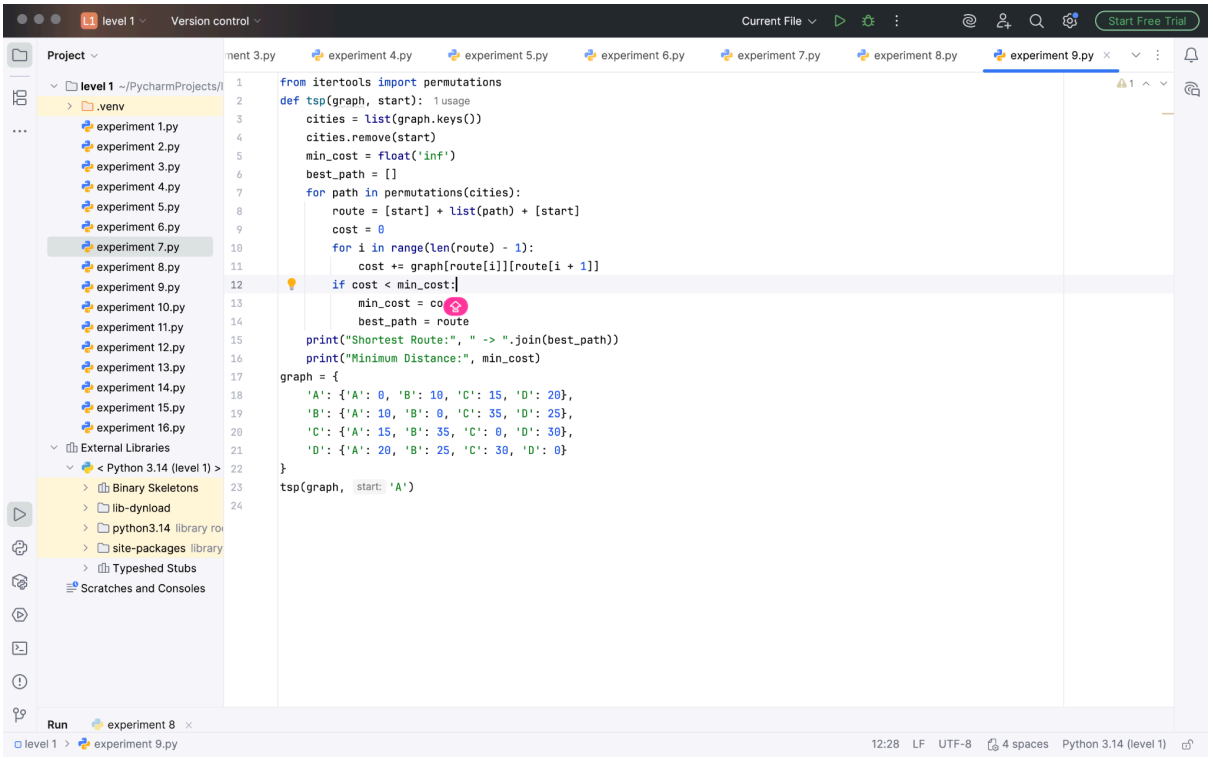
Aim

To write a Python program to solve the Travelling Salesman Problem by finding the shortest possible route that visits every city exactly once and returns to the starting city.

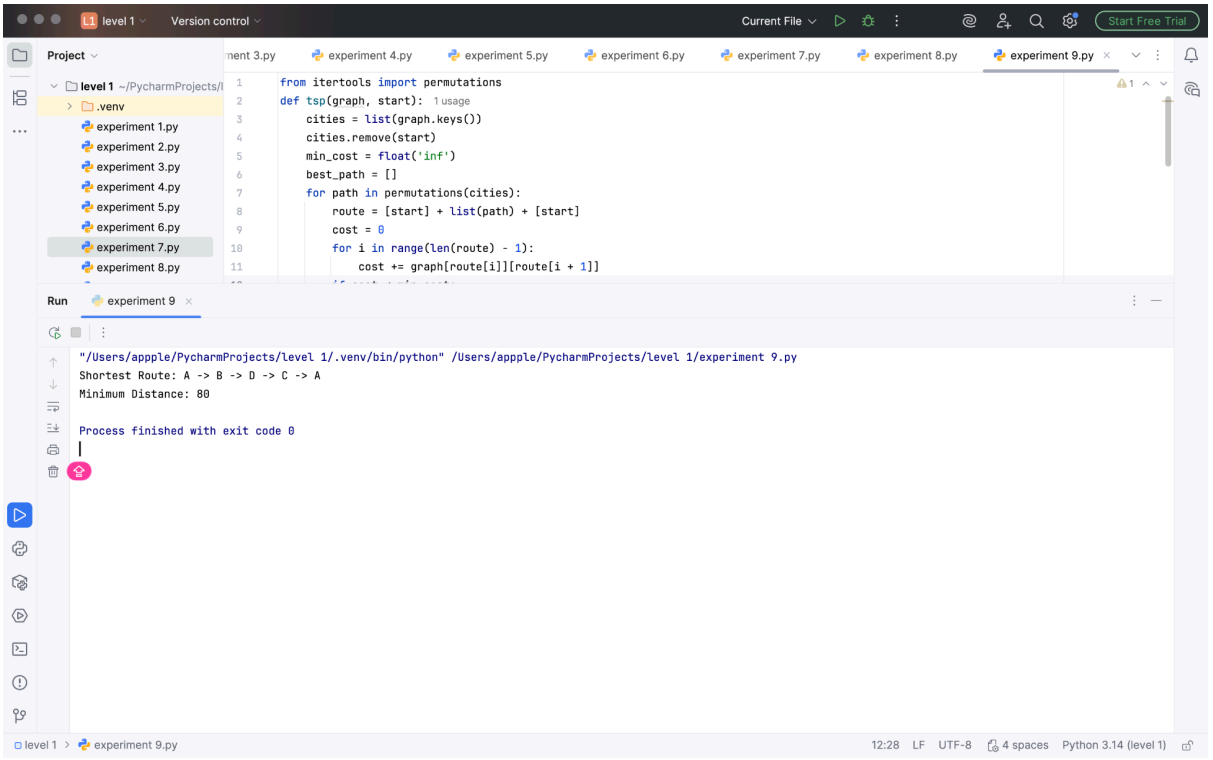
Algorithm

1. Start the program.
2. Define the cities and the distance matrix.
3. Generate all possible routes between the cities.
4. Calculate the total distance of each route.
5. Select the route with the minimum total distance.
6. Display the shortest route and its total distance.
7. Stop the program.

CODE:



OUTPUT:



RESULT:

Thus, the Python program successfully solves the Travelling Salesman Problem and finds the shortest route with a minimum distance of 80 units.

PROGRAM 10: Python program to implement A* algorithm

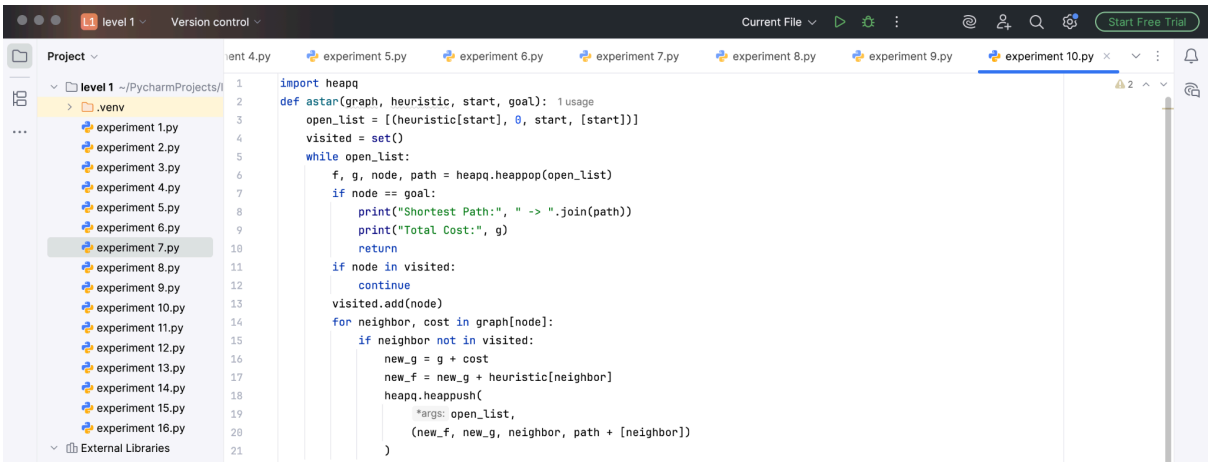
Aim

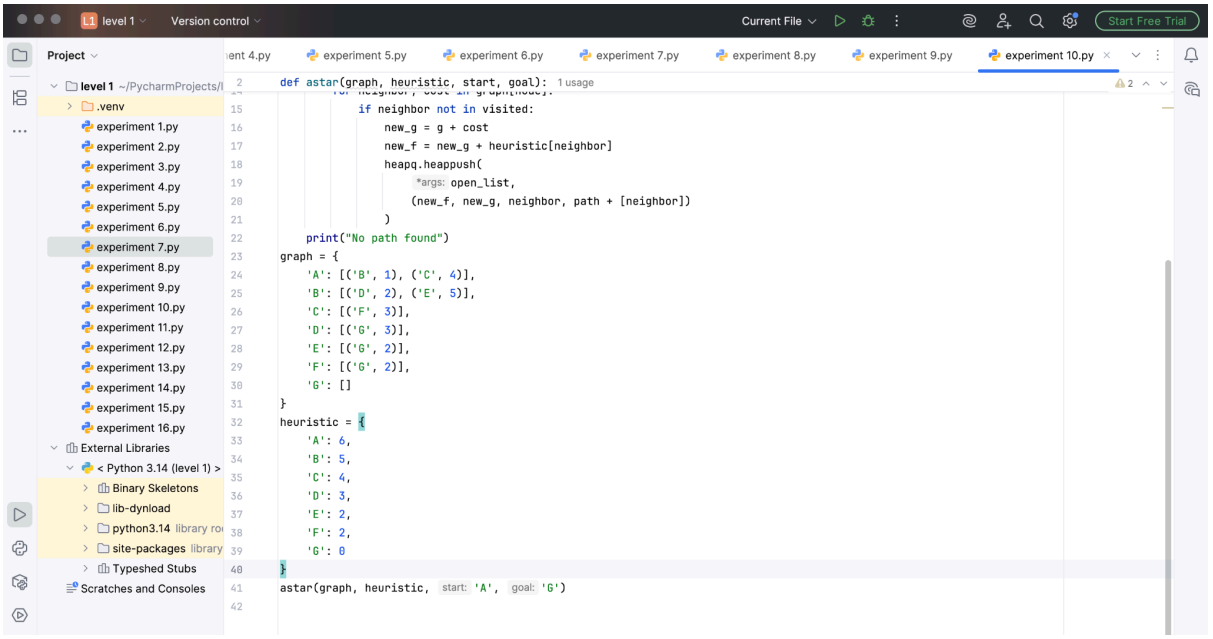
To write a Python program to implement the A* search algorithm to find the shortest path from a starting node to a goal node using path cost and heuristic cost.

Algorithm

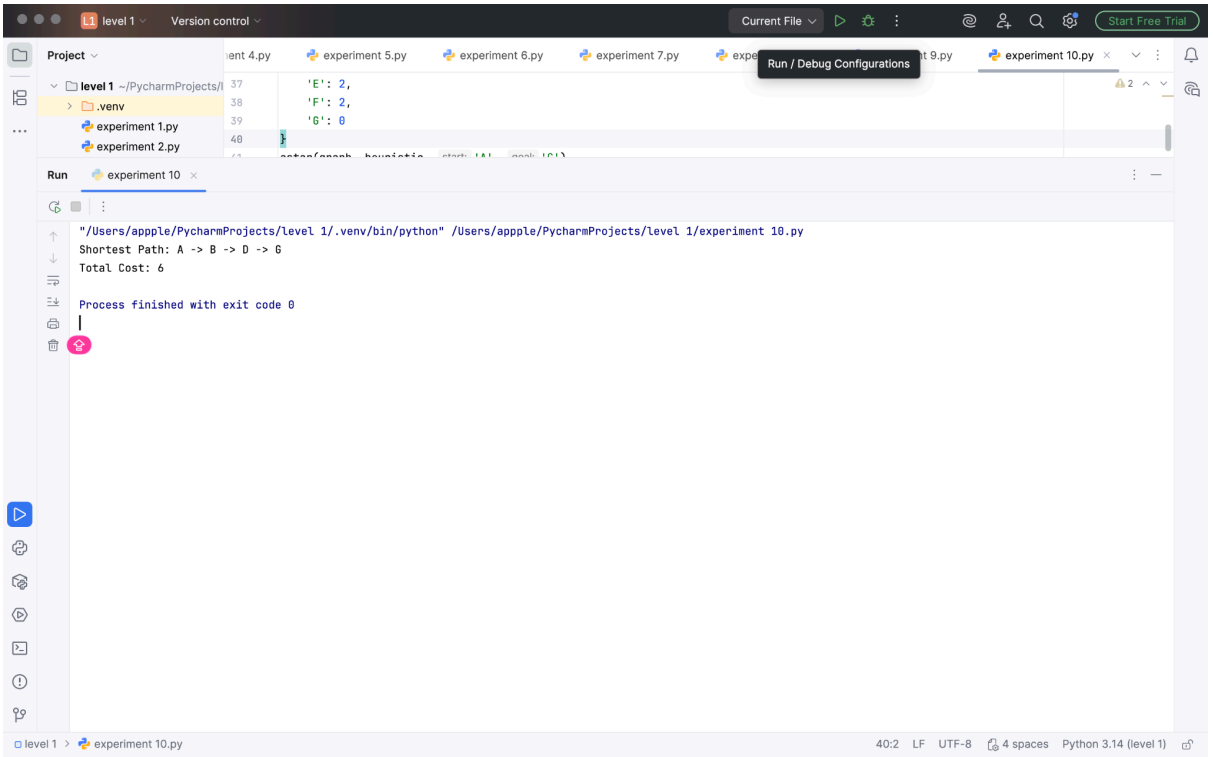
1. Start the program.
2. Define the graph with edge costs and heuristic values.
3. Initialize the open list with the starting node.
4. Calculate $f(n) = g(n) + h(n)$, where $g(n)$ is the path cost and $h(n)$ is the heuristic cost.
5. Select the node with the lowest $f(n)$ value.
6. If the selected node is the goal, display the shortest path and total cost.
7. Otherwise, explore its neighboring nodes and update their costs.
8. Repeat until the goal is reached or no path exists.
9. Stop the program.

CODE:





OUTPUT:



RESULT:

Thus, the Python program successfully implements the A* search algorithm and finds the shortest path from the starting node A to the goal node G with a total cost of 6.